

Predicting Standing Waves on a Square Plate

Michael Xu

May 30, 2026

Abstract

This investigation aims to use a computer program to predict the Chladni setup for a square metal plate. The plate is vibrated from the middle using function generator. Standing wave patterns on the plate are observed for multiple frequencies. A Python program is written and aims to simulate this setup and predict the patterns. Multiple algorithms are tried and non of them can produce patterns which look identical to the experiment. However, some algorithms predicted certain frequencies really well. Further improvements of the algorithm with better hardwares can be done to obtain more accurate results in the future.

1 Introduction

This research aims to predict the node patterns formed on a vibrating square metal plate. Just like how standing waves can be formed on a string, it can also form on a 2D surface when the plate is vibrating at a certain frequency. This was first demonstrated in public by a German physicist and musician, Ernst Florens Friedrich Chladni.¹ In his original setup, he fixed a metal plate in the centre and used a violin bow to vibrate the edge of the plate. At certain frequencies, standing waves will be formed on the plate. Chladni visualised this by sprinkling sand on to the plate. The antinodes oscillates which displaces the sand to the nodes, which are still. The sand accumulates in the nodes which forms impressive patterns on the plate. This is later known as the "Chladni Setup".

Chladni Setup is recreated in this experiment by fixing the centre of the plate to an oscillator which is controlled by a function generator. The frequency and the amplitude of the vibration can be controlled directly using the function generator.

This investigation tries to predict the patterns formed on a square plate by using different algorithms explained in the next section.

2 Theory

2.1 The 2D Wave Equation

The first algorithm treats the metal plate as a thin membrane. It has negligible thickness and no elasticity. The waves on the plate is modeled as if they are vibrating freely, with no damping. Therefore, the waves on the plate simply satisfies the 2D wave equation.² This is the simplest model that can be constructed to simulate the Chladni plates.

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u \quad (1)$$

The symbol u is a function of t , time, and x, y which are Cartesian coordinates. It represents the vertical displacement of the wave at the given position and time. The ∇^2 symbol is the 2D laplacian operator. The quantity c is the wave speed.

When the wave is a standing wave on the plate, we can assume that the time and space sections of u is separable, $u(x, y, t) = F(x, y)T(t)$, with the time dependent part $T(t) = \cos(\omega t)$. Also for convenience, x, y , and t will be casted in dimensionless form. Time t will be multiplied by $\frac{L}{c}$. x and y will all be divided by L , and u represents a fraction of its amplitude. All of the quantities in the paper will be expressed in this dimensionless form later on.

In this way, the wave equation can be rewritten into the time independent form:

$$\nabla^2 u = -k^2 u \quad (2)$$

This is essentially a differential equation and one is trying to find the eigenfunctions of the laplacian operator. One can obtain ω by multiplying k with $\frac{c}{L}$.

An important detail for this PDE is it's boundary conditions. Normally when solving this type of wave equations for a vibrating string, or the Schrodinger's Wave Equation, a Dirichlet boundary condition is assumed. This means that $u = 0$ when $x = 0$ or $x = 1$. This corresponds to the case where the wave must disappear at the edges, i.e. clamped strings or infinite potential. However, for a vibrating plate, the edges are allowed to vibrate freely, often vibrating at a maximum or minimum. Therefore, instead of $u(0) = 0$, it is $\frac{du}{dx} \big|_{x=0} = 0$ (Also the same for $x = 1$). This is called as the Neumann boundary condition. Both boundary

¹Linqi 2018.

²Emmanuel 2010.

conditions are tested in the practical for the this investigation

Therefore, using this boundary condition, the general solution of this PDE is this:

$$u(x, y, t) = U_0 \cos(\omega_{ms} t) \cos(m\pi x) \cos(s\pi y) \quad (3)$$

U_0 is the initial displacement (i.e amplitude) of the wave. m and s are the indices for the different modes. ω_{ms} is the angular frequency of the wave and is related to m and s via this relationship: $\omega_{ms} = \sqrt{m^2 + s^2}$. Note that a value of ω can correspond to multiple combinations of m and s . This means that in the practical, the overall pattern displayed is going to be a superposition of the different modes.

Another important feature of this solution is that each mode is orthogonal to each other. This is key thing to remember when dealing with the discretised version of the laplacian, especially when considering the boundary conditions.

2.2 Numerically Solving the Wave Equation

Analytical solutions may be convenient for certain situations, however, not any shape has an analytical solution. Sometimes, numerical methods are required. The finite differences method (FDM) can be used to solve this PDE numerically. Dividing the region by N , dx can be turned into Δx , with $N\Delta x = 1$. This means that the function $u(x)$ is also discretised into u_i where i is an integer ranges from 0 to N . Only one dimension is considered in this situation as it simplifies things a lot. The method to raise this to 2D will be shown later.

2.2.1 Dirichlet Boundary Condition

Using the central difference approach, the second derivative of a point u_i can be thought as the rate of change between the derivative of the imaginary point $u_{i+0.5}$ and the imaginary point $u_{i-0.5}$.

$$\frac{\Delta^2 u_i}{\Delta x^2} = \left(\frac{\Delta u_{i+0.5}}{\Delta x} - \frac{\Delta u_{i-0.5}}{\Delta x} \right) \frac{1}{\Delta x} \quad (4)$$

As the derivatives of $u_{i+0.5}$ and $u_{i-0.5}$ can be defined like this,

$$\frac{\Delta u_{i+0.5}}{\Delta x} = \frac{u_{i+1} - u_i}{\Delta x}; \quad \frac{\Delta u_{i-0.5}}{\Delta x} = \frac{u_i - u_{i-1}}{\Delta x} \quad (5)$$

the laplacian operator can be rewritten like this:

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2} = -\omega^2 u_i \quad (6)$$

Or in a matrix form:

$$\begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \begin{pmatrix} u_{i+1} \\ u_i \\ u_{i-1} \end{pmatrix} = -\Delta x^2 \omega^2 u_i \quad (7)$$

Because the different u_i can be grouped into one vector like this:

$$\mathbf{u} = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{N-1} \end{pmatrix} \quad (8)$$

the matrix equation can therefore be rewritten again into this form:

$$\begin{pmatrix} -2 & 1 & 0 & 0 & \cdots & 0 \\ 1 & -2 & 1 & 0 & & \vdots \\ 0 & 1 & -2 & 1 & & \vdots \\ 0 & 0 & 1 & \ddots & \ddots & 0 \\ \vdots & & & \ddots & \ddots & 1 \\ 0 & \cdots & \cdots & 0 & 1 & -2 \end{pmatrix} \mathbf{u} = -\Delta x^2 \omega^2 \mathbf{u} \quad (9)$$

Notice that u_0 and u_N are both neglected in this vector as they are all 0 in this boundary condition.

This matrix equation turns the PDE into a simpler eigenvalue and eigenvector problem. Notice that the coefficient $-\Delta x^2 \omega^2$ is simply the eigenvalue and $\mathbf{u}$ is the eigenvector of the laplacian matrix. The different eigenvector solutions in this equation is also orthogonal (where the inner product is the dot product instead of the integral). The meaning is that each eigenvector is perpendicular to each other in their ultra high dimensional space ($N-1$ dim in this case). This is exactly the same ideas as the orthogonal property existing in the continuous, analytical solutions of the equation.

A computer can easily calculate the eigenvector and eigenvalue of this diagonal matrix given enough time and memory.

2.2.2 Neumann Boundary Condition

The matrix shown previously is the discrete Dirichlet Laplacian. The case for free edges is the discrete Neumann Laplacian. As mentioned before, Neumann boundary condition assumes zero flux, i.e. $\frac{du}{dx} = 0$, for $x = 0$ or $x = 1$. When discretised, a ghost point (non existent but imagined) u_{-1} or u_{N+1} can be used to represent the derivative.

$$\frac{\Delta u_0}{\Delta x} = \frac{u_1 - u_{-1}}{2\Delta x} = 0; \quad \frac{\Delta u_N}{\Delta x} = \frac{u_{N+1} - u_N}{2\Delta x} = 0 \quad (10)$$

This implies that $u_1 = u_{-1}$ and $u_N = u_{N+1}$. By applying this to (6), this means that the second derivative of the boundaries is this:

$$\frac{u_1 - 2u_0 + u_{-1}}{\Delta x^2} = \frac{2u_1 - 2u_0}{\Delta x^2} = -\omega^2 u_0 \quad (11)$$

$$\frac{u_{N+1} - 2u_N + u_{N-1}}{\Delta x^2} = \frac{-2u_N + 2u_{N-1}}{\Delta x^2} = -\omega^2 u_0 \quad (12)$$

The change to the original matrix equation is that the first row is $(-2 \ 2 \ 0 \ \cdots \ 0)$ and the last row is $(0 \ \cdots \ 0 \ 2 \ -2)$. The matrix also changes from $N-1$ by $N-1$ to $N+1$ by $N+1$. The full matrix equation is shown below:

$$\begin{pmatrix} -2 & 2 & 0 & 0 & \cdots & 0 \\ 1 & -2 & 1 & 0 & & \vdots \\ 0 & 1 & -2 & 1 & & \vdots \\ 0 & 0 & 1 & \ddots & \ddots & 0 \\ \vdots & & & \ddots & \ddots & 1 \\ 0 & \cdots & \cdots & 0 & 2 & -2 \end{pmatrix} \mathbf{u} = -\Delta x^2 \omega^2 \mathbf{u} \quad (13)$$

Notice that the vector $\mathbf{u}$ now starts with u_0 and ends with u_N . There is a problem for this matrix though. The Laplacian operator is a Hermitian operator which gives it the feature of orthogonal eigenfunctions. The corresponding matrix must also be a Hermitian matrix. This means that the complex conjugate of the transpose of the matrix must be the same as the original matrix. As the matrix shown above is real, this means that the discrete laplacian must have a reflectional symmetry through its main diagonal. The matrix shown above is clearly asymmetrical through the main diagonal. This is not a huge problem mathematically but does put a lot of computational pressure on the computer when it is trying to solve for the eigenvalues and eigenvectors.

To avoid this problem, forward/backward differences can be used instead of central differences method when calculating the second derivatives for the boundaries. Hence, instead of applying the equation shown in (4), one should use this for the left boundary (forward difference):

$$\nabla^2 u_0 = \left(\frac{\Delta u_1}{\Delta x} - \frac{\Delta u_0}{\Delta x} \right) \frac{1}{\Delta x} \quad (14)$$

and this for the right boundary (backward difference):

$$\nabla^2 u_N = \left(\frac{\Delta u_N}{\Delta x} - \frac{\Delta u_{N-1}}{\Delta x} \right) \frac{1}{\Delta x} \quad (15)$$

Using the definition of the Neumann boundary condition, the $\frac{\Delta u_0}{\Delta x}$ term and the $\frac{\Delta u_N}{\Delta x}$ all equals to 0. When backward difference and forward difference is then applied to the left over first derivative terms in the equations respectively, the two second derivatives can be written as follow:

$$\nabla^2 u_0 = \frac{-u_0 + u_1}{\Delta x^2}; \nabla^2 u_N = \frac{u_{N-1} - u_N}{\Delta x^2} \quad (16)$$

Therefore, the overall discrete Neumann laplacian can be expressed as shown:

$$\begin{pmatrix} -1 & 1 & 0 & 0 & \cdots & 0 \\ 1 & -2 & 1 & 0 & & \vdots \\ 0 & 1 & -2 & 1 & & \vdots \\ 0 & 0 & 1 & \ddots & \ddots & 0 \\ \vdots & & & \ddots & \ddots & -2 \\ 0 & \cdots & \cdots & 0 & 1 & -1 \end{pmatrix} \mathbf{u} = -\Delta x^2 \omega^2 \mathbf{u} \quad (17)$$

In this way, the diagonally symmetrical (Hermitian) nature of the laplacian matrix is preserved which made it much cleaner to calculate for the eigenvectors and eigenvalues.

2.2.3 Raising to 2D

The discretised laplacian matrices mentioned previously are all in 1 dimension. This is done intentionally as it is much easier to work with. However, 1D laplacian might be able to model oscillations on a violin string, it is unable to model the 2D waves on a plate. The laplacian must be transformed to its 2D form to model the Chladni setup correctly.

The expanded form of the 2D laplacian is shown:

$$\nabla^2 u = \frac{\partial u}{\partial x} + \frac{\partial u}{\partial y} \quad (18)$$

By imagining a L by L space and dividing the space into a N by N mesh, each discrete point on the mesh can be represented by this notation $u_{i,j}$ where i and j both ranges from 0 to N. As the space is divided into a square mesh, a quantity Δh can be defined so that $\Delta h = \Delta x = \Delta y$. Therefore, using the equations derived previously, the 2 dimensional second derivative can be expressed as following:

$$\frac{u_{i,j+1} + u_{i+1,j} - 4u_{i,j} + u_{i-1,j} + u_{i,j-1}}{\Delta h^2} = -\omega^2 u_{i,j} \quad (19)$$

Flattening the 2 dimensional array of $u_{i,j}$ into 1 column of vector $\mathbf{u}$, the matrix equation of the Dirichlet boundary condition is this:

$$M\mathbf{u} = -\Delta x^2 \omega^2 \mathbf{u} \quad (20)$$

where the vector $\mathbf{u}$ is

$$\mathbf{u} = \begin{pmatrix} u_{1,1} \\ \vdots \\ u_{1,N-1} \\ u_{2,1} \\ \vdots \\ u_{2,N-1} \\ \vdots \\ u_{N-1,N-1} \end{pmatrix} \quad (21)$$

M is a $(N-1)^2 \times (N-1)^2$ tridiagonal matrix

$$M = \begin{pmatrix} B & I & 0 & 0 & \cdots & 0 \\ I & B & I & 0 & & \vdots \\ 0 & I & B & I & & \vdots \\ 0 & 0 & I & \ddots & \ddots & 0 \\ \vdots & & & \ddots & B & I \\ 0 & \cdots & \cdots & 0 & I & B \end{pmatrix} \quad (22)$$

with B being another tridiagonal matrix of size $(N-1) \times (N-1)$ that looks like this:

$$B = \begin{pmatrix} -4 & 1 & 0 & 0 & \cdots & 0 \\ 1 & -4 & 1 & 0 & & \vdots \\ 0 & 1 & -4 & 1 & & \vdots \\ 0 & 0 & 1 & \ddots & \ddots & 0 \\ \vdots & & & \ddots & -4 & 1 \\ 0 & \cdots & \cdots & 0 & 1 & -4 \end{pmatrix} \quad (23)$$

and I is the identity matrix of the same shape with B

The 2 dimensional discrete Neumann Laplacian has a similar equation but with $\mathbf{u}$ being

$$\mathbf{u} = \begin{pmatrix} u_{0,0} \\ u_{0,1} \\ \vdots \\ u_{0,N} \\ u_{1,0} \\ \vdots \\ u_{1,N} \\ \vdots \\ u_{N,N} \end{pmatrix} \quad (24)$$

M is now a $(N+1)^2 \times (N+1)^2$ tridiagonal matrix that looks like this

$$M = \begin{pmatrix} B_1 & I & 0 & 0 & \cdots & 0 \\ I & B_2 & I & 0 & & \vdots \\ 0 & I & B_2 & I & & \vdots \\ 0 & 0 & I & \ddots & \ddots & 0 \\ \vdots & & & \ddots & B_2 & I \\ 0 & \cdots & \cdots & 0 & I & B_1 \end{pmatrix} \quad (25)$$

with B_1 and B_2 displayed below respectively.

$$B_1 = \begin{pmatrix} -2 & 1 & 0 & 0 & \cdots & 0 \\ 1 & -3 & 1 & 0 & & \vdots \\ 0 & 1 & -3 & 1 & & \vdots \\ 0 & 0 & 1 & \ddots & \ddots & 0 \\ \vdots & & & \ddots & -3 & 1 \\ 0 & \cdots & \cdots & 0 & 1 & -2 \end{pmatrix} \quad (26)$$

$$B_2 = \begin{pmatrix} -3 & 1 & 0 & 0 & \cdots & 0 \\ 1 & -4 & 1 & 0 & & \vdots \\ 0 & 1 & -4 & 1 & & \vdots \\ 0 & 0 & 1 & \ddots & \ddots & 0 \\ \vdots & & & \ddots & -4 & 1 \\ 0 & \cdots & \cdots & 0 & 1 & -3 \end{pmatrix} \quad (27)$$

These matrices might look horrible at first glance but are actually surprisingly easy to construct. Defining the original 1 dimensional matrix as D . Matrix M is simply the kronecker sum³ of 2 D s, with the respective boundary conditions.

$$M = D \oplus D \quad (28)$$

The symbol $\oplus$ denotes the kronecker sum and can be expressed as this

$$D \oplus D = D \otimes I + I \otimes D \quad (29)$$

where the symbol $\otimes$ is the kronecker product of a matrix. The exact detail and proof of this will not be shown in here.

2.3 Forced Wave Equation

The previous algorithm simulates free oscillations on a 2D membrane. It is relatively straightforward to calculate and is the easiest to implement. However, this is not what happens exactly for the experiment. There is a constant source of forced vibration in the middle of the plate, driving the standing waves. Therefore, instead of the simple free oscillation equation, a periodic forced term should also be added to the equations as shown:

$$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u + F(x, y, t) \quad (30)$$

Where $F(x, y, t)$ is the forced term (to simulate the constant oscillation of the wave generator) and takes the form:

$$F(x, y, t) = \begin{cases} \alpha \cos(\omega t) & \text{if } x = y = 0 \\ 0 & \text{otherwise} \end{cases} \quad (31)$$

ω represents the angular frequency of the vibration generator. α is the amplitude of the oscillation. This is essentially a Dirac Delta function in both x and y directions that is multiplied by the time evolution term:

$$F(x, y, t) = \alpha \cos(t) \delta(x) \delta(y) \quad (32)$$

In this case, the plate centre is $(0, 0)$ and x and y ranges from -1 to 1. The two Dirac Delta functions models the central point on the plate where the plate receives the constant driving force of oscillation. The contact point is modeled as an infinitesimally small, fixed point, as if the vibration generator is connected to the plate via a small bolt. If the frequency of the forced term matches one of the modes calculated from the previous algorithm, the amplitude of the oscillation should diverge to infinity and the pattern should stabilise as time evolves.

³Loan 2000.

2.4 Numerically Solving the Forced Equation

The numerical solution of the forced equation is very similar to the free wave equation. The domain is still divided into an uniform $(N+1) \times (N+1)$ space with spacing $\Delta x = \Delta y = \Delta h = \frac{2}{N}$. Notice that the mesh grid ranges from -1 to 1 hence the respective discrete coordinates are written as $x_i = -1 + i\Delta h$ and $y_j = -1 + j\Delta h$, where i and j ranges from 0 to N . Another difference is that because of the time dependent nature of forcing term, time must also be included and discretised, with $\Delta t = \frac{1}{N}$ and $t_n = \Delta t n$ where n ranges from 0 to N . With the spacial and time coordinates discretised, the displacement function u can now be written as $u_{i,j}^n$.

The 2D laplacian in the forced wave equation is still expressed as the standard five-point finite difference stencil as before:

$$\nabla^2 u_{i,j}^n = \frac{u_{i,j+1}^n + u_{i+1,j}^n - 4u_{i,j}^n + u_{i-1,j}^n + u_{i,j-1}^n}{\Delta h^2} \quad (33)$$

The new included second derivative with respect to time is also approximated using the central difference method:

$$\frac{\partial^2 u_{i,j}^n}{\partial t^2} = \frac{u_{i,j}^{n+1} - 2u_{i,j}^n + u_{i,j}^{n-1}}{\Delta t^2} \quad (34)$$

Using the approximations of the derivatives, the forced wave equation can be substituted and rearranged to look like this:

$$u_{i,j}^{n+1} = 2u_{i,j}^n - u_{i,j}^{n-1} + \Delta t^2 (c^2 \nabla^2 u_{i,j}^n + F(x_i, y_j, t_n)) \quad (35)$$

With the initial condition $u_{i,j}^0$ and $u_{i,j}^1$ predetermined, $0 \leq i, j \leq N$, the formula can be implemented both iteratively and recursively in code to calculate the next u in space and time. This scheme is known as the leapfrog method⁴ and is both time reversible and energy-conserving in an undamped system. It requires no linear systems, but rather simple arithmetic between the previous two terms which makes this both easy to implement and fast to calculate.

2.4.1 Boundary Conditions

At the edges of the system, when the i and j indices go out of bounds, one can let $u_{-1,j}$ and $u_{N+1,j}$ be 0 for Dirichlet boundary condition. For Neumann conditions, the out-of-bound terms will be reflected as derived previously: $u_{-1,j} = u_{1,j}$ and $u_{N+1,j} = u_{N-1,j}$. The same rules apply also to j .

2.5 Kirchhoff-Love Plate Equation

The previous two algorithms both model the waves vibrating on a non elastic membrane. They are simple

models but will produce patterns that are not as accurate. To properly govern dynamics of a metal plate, the Kirchhoff-Love plate equation should be used^{5,6}. It is derived by balancing the external loads with the internal bending force that tends to restore the plate to its stress-free state. This paper considers a plate model that is generalized from the classical Kirchhoff-Love equation by including additional terms to account for more physical effects, such as linear restoration, tension and visco-elasticity.⁷

The full equation for a plate with thickness h is shown below. Some parts of the equation are grouped and combined into a single term for simplicity reasons.

$$\rho h a(x, y, t) = -\hat{K}u(x, y, t) - \hat{B}v(x, y, t) + F(x, y, t) \quad (36)$$

Here, $\hat{K}$ is the internal forces operator and $\hat{B}$ is the damping operator.

$$\hat{K} = K_0 - T\nabla^2 + D\nabla^4 \quad (37)$$

$$\hat{B} = K_1 - T_1\nabla^2 \quad (38)$$

$u(x, y, t)$ still represents the vertical displacement of the wave. $a(x, y, t)$ and $v(x, y, t)$ is the syntactic sugar for $\frac{\partial^2 u}{\partial t^2}$ and $\frac{\partial u}{\partial t}$ respectively.

The other symbols in this equation not mentioned are constants describing the material properties of the plate.

Symbol	Constant	Unit
ρ	Density	kg m ⁻³
K_0	Linear Stiffness Coefficient	kg m ⁻² s ⁻²
T	Tension Coefficient	kg s ⁻²
D	Flexural Rigidity	Nm
K_1	Linear Damping Term	kg m ⁻² s ⁻¹
T_1	Visco-Elastic Damping Term	kg s ⁻¹

Note that $D = \frac{Eh^3}{12(1-\nu)}$ where h is the plate thickness, E is the Young's modulus, and ν is the Poisson's ratio.

The ∇^4 term is called as the Biharmonic operator. It is equals to the square of the laplacian.

The $F(x, y, t)$ term is still the forcing term created by the vibration generator. To pursue for a more accurate approximation, the forcing term will now be applied to a small range in x and y rather than the central point.

$$F(\mathbf{x}, t) = \begin{cases} \alpha \cos(\omega t), & \mathbf{x} \in A \\ 0, & \mathbf{x} \notin A \end{cases} \quad (39)$$

$\mathbf{x}$ denotes the spacial vector representing x and y . A represents a square area where $A = [-0.001, 0.001] \times [-0.001, 0.001]$. This still mimics a bolt fixing the plate to a vibration generator in the middle but the bolt now

⁴Strang 2006.

⁵Kirchhoff 1850.

⁶Love 1888.

⁷Nguyen, Li, and Ji 2021.

has a length and a width instead of being a singular point.

2.6 Numerically Solving Kirchhoff-Love Plate Equation

The numerical algorithm used to solve this PDE is called the Newmark-Beta Scheme (NB2)^{8,9}. Note that the 2D space is discretised in the same fashion as in Section 2.4. The spacial indices i and j are combined into the general index $\mathbf{i}$ for simplicity in notation.

The algorithm first defines the relationship between the acceleration, velocity, and displacement using the Taylor expansion. Given the acceleration of the next time step a_i^{n+1} , the displacement and the velocity at the next time step can be solved using the quantities from the previous time step:

$$u_i^{n+1} = u_i^n + \Delta t v_i^n + \frac{\Delta t^2}{2} \frac{(a_i^{n+1} + a_i^n)}{2} \quad (40)$$

$$v_i^{n+1} = v_i^n + \Delta t \frac{(a_i^{n+1} + a_i^n)}{2} \quad (41)$$

Given that the two relationships is a linear combination of terms from two time steps, the equations can be separated into:

$$u_i^{n+1} = u_i^p + \frac{\Delta t^2}{4} a_i^{n+1} \quad (42)$$

$$v_i^{n+1} = v_i^p + \frac{\Delta t}{2} a_i^{n+1} \quad (43)$$

where $u_i^p = u_i^n + \Delta t v_i^n + \frac{\Delta t^2}{4} a_i^n$ is the predicted value for u and $v_i^p = v_i^n + \frac{\Delta t}{2} a_i^n$ is the predict value for v .

The acceleration for the next time step is defined separately by the plate equation stated previously:

$$\rho h a_i^{n+1} = -\hat{K} u_i^{n+1} - \hat{B} v_i^{n+1} + F_i^{n+1} \quad (44)$$

Note that the operators are all discretised using similar methods explained previously. Substitute the quantities and rearranging yields this final time stepping equation to calculate for each a_i^{n+1} iteratively:

$$(\rho h + \frac{\Delta t^2}{4} \hat{K} + \frac{\Delta t}{2} \hat{B}) a_i^{n+1} = -\hat{K} u_i^p - \hat{B} v_i^p + F_i^{n+1} \quad (45)$$

After knowing a_i^{n+1} , u_i^{n+1} and v_i^{n+1} can all be found by using the equation (40) and equation (41).

2.6.1 Boundary Condition

The boundary conditions for this case is a bit more complicated than the simple wave equation case. For the free edge, net zero bending force and zero shear force on the edge must be ensured^{10,11}. Exact derivation won't be shown in the paper due to limiting spaces but this leads to the followings for the edge and corners of the plate: $u_{-1,j} = 2u_{0,j} - u_{1,j}$ and $u_{-1,-1} = u_{-1,1} + u_{1,-1} - u_{1,1}$.¹² Only one edge and one corner is shown in here but the same applies to all four edges and corners.

3 Methodology

3.1 Chladni Plates

The physical experiment is conducted by attaching a metal plate to a wave generator. The wave generator controls the frequency and amplitude of the vibration. Pouring sand onto the plate causes the sand to accumulate on the nodes (zero displacement) when the frequency is tuned correctly. The frequencies tuned to are those that maximise the volume of the sound produced due to the vibration. The nodal patterns can be visualised in this way. The apparatus used is shown below.

The procedure described above corresponds to the free edge (Neumann) boundary. For the Dirichlet boundary condition, the edges of the plate must be clamped. Wooden frame is manufactured to the dimensions of the plate and secured to the working platform using a G-clamp. However, it is noticed that due to the clamping, the stress on the plate is increased. This makes the piston increasingly hard to cause any notable vibrations on the plate. Sand is too coarse and too heavy to visualise the patterns. To overcome this issue, the surface of the plate is cleaned carefully to make it smooth as possible. Sand is also replaced by finer, and less dense sugar. The setup is shown.

Plates of different shapes are also tested. A circular plate is tested for different patterns and a smaller square plate is tested to look for the change in frequency.

3.2 Models

For the analytical solutions of the square plate, the equations are plotted using Desmos.com. The corresponding modes for integer values of m and s are all compared with the patterns observed experimentally.

Numerical methods explained previously are implemented in Python (python 3.12). See Appendix,

⁸Rao 2011.

⁹Newmark 1959.

¹⁰Kirchhoff 1850.

¹¹Timoshenko and Woinowsky-Krieger 1959.

¹²Nguyen, Li, and Ji 2021.

¹³Numpy Documentation n.d.

¹⁴Scipy Documentation n.d.

¹⁵Matplotlib Documentation n.d.

or the Github repository (<https://github.com/michael-xu-wenmo/Physics-Personal-Investigation>) for the full implementation. External Python libraries like Numpy¹³ and Scipy¹⁴ are used when implementing the algorithms to simplify the codebase. The library Matplotlib¹⁵ is also used to visualise the patterns generate and plot the frequency graphs.

For algorithms that are also dependent on time (forced wave equation), a large time period ($T = 100$) will be chosen to let the waves propagate and stabilise into a standing wave and the final pattern at $T = 100$ will be saved to use for comparison.

It is important to aware that the experimental results are usually a superposition of different degenerate modes. When visualising the eigenvectors, degenerate eigenvectors will also be superposed together to look for more accurate predictions.

4 Results

The patterns from the free edge Chladni setup for the square plate are shown below and labeled with their corresponding frequencies.

The patterns for the clamped edge Chladni setup are not shown because the apparatus failed to visualise them. When the edges are clamped. The tension in the plate is too high to make any significant vibrations. The wave generator used cannot create waves with enough amplitude to create any notable waves. Furthermore, the frequencies required to create the waves are also increased. At high frequencies, the wave generator cannot create vibrations with precise enough frequencies. This is why the Chladni Setup failed to visualise clamped edge situations in this investigation.

4.1 Wave Equations

The analytical and the numerical solutions of the wave equation is shown below.

The patterns of the analytical and numerical solutions matches each other perfectly at every modes. However, when compared to the actual Chladni waves, there are still some notable differences.

These differences comes from multiple factors and is due to the simplicity of this model. It haven't taken the material properties of the metal plate into account and models waves under free oscillation when they are actually driven by an external source.

The frequencies predicted by this model are all arbitrary and should not be compared to the actual frequencies in the experiment. This is because this model assumes that the waves exists on a non-elastic membrane with negligible thickness. The wave speed therefore is not taken into account.

4.2 Forced Wave Equation

5 Conclusion

The Chladni setup for a square plate is conducted in a laboratory and different algorithms are conducted to simulate the patterns created.

The first algorithm models the standing waves on the plate as if they exists on a 2D membrane and oscillates freely with no damping. This allows the waves to be governed by the 2D wave equation. Both analytical and numerical solutions are implemented to simulate the standing waves. This model creates some similar looking patterns to the actual ones at certain modes but lacks in details.

References

- Emmanuel, DiBenedetto (2010). *Partial Differential Equations*. 2nd ed. Birkhauser Boston Inc.
- Kirchhoff, Gustav (1850). “Über das Gleichgewicht und die Bewegung einer elastischen Scheibe”. German. In: *Journal für die reine und angewandte Mathematik* 40, pp. 51–88. DOI: 10.1515/crll.1850.40.51. URL: <https://doi.org/10.1515/crll.1850.40.51>.
- Linqi, Shao (2018). “Modeling a Square Vibrating Plate”. In: URL: <https://www.semanticscholar.org/paper/Modeling-a-Square-Vibrating-Plate-Shao/e5dbf068ca45b54ec7134794dc0ef73fa9893741>.
- Loan, Charles F. Van (2000). “The Ubiquitous Kronecker Product”. In: *Journal of Computational and Applied Mathematics* 123.1–2, pp. 85–100. DOI: 10.1016/S0377-0427(00)00393-9.
- Love, Augustus Edward Hough (1888). “The Small Free Vibrations and Deformation of a Thin Elastic Shell”. In: *Philosophical Transactions of the Royal Society of London A* 179, pp. 491–546. DOI: 10.1098/rsta.1888.0016. URL: <https://doi.org/10.1098/rsta.1888.0016>.
- Matplotlib Documentation* (n.d.). URL: <https://matplotlib.org/stable/>.
- Newmark, Nathan M. (1959). “A Method of Computation for Structural Dynamics”. In: *Journal of the Engineering Mechanics Division* 85.EM3, pp. 67–94. DOI: 10.1061/JMCEA3.0000098.
- Nguyen, Duong T. A., Longfei Li, and Hangjie Ji (2021). “Stable and Accurate Numerical Methods for Generalized Kirchhoff–Love Plates”. In: *Journal of Engineering Mathematics* 130.6. DOI: 10.1007/s10665-021-10163-x. URL: <https://doi.org/10.1007/s10665-021-10163-x>.
- Numpy Documentation* (n.d.). URL: <https://numpy.org/devdocs/index.html>.
- Rao, Singiresu S. (2011). *The Finite Element Method in Engineering*. 5th ed. Boston: Butterworth-Heinemann. ISBN: 9781856176613. URL: <https://www.sciencedirect.com/book/9781856176613/the-finite-element-method-in-engineering>.
- Scipy Documentation* (n.d.). URL: <https://scipy.org/>.
- Strang, Gilbert (2006). *The Wave Equation and Staggered Leapfrog*. MIT 18.086 Mathematical Methods for Engineers II lecture notes. Massachusetts Institute of Technology. URL: <https://math.mit.edu/classes/18.086/2006/am53.pdf> (visited on 05/29/2026).
- Timoshenko, Stephen P. and S. Woinowsky-Krieger (1959). *Theory of Plates and Shells*. 2nd ed. New York: McGraw-Hill.